redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

Team Name : SavioMohan1

Problem Statement : Intelligent Candidate Discovery and Ranking

Proposed solution: an offline, explainable ranker that reads the Senior AI Engineer JD and scores all 100,000 candidates into a trusted top-100 shortlist.

The system focuses on recruiter-grade evidence: production retrieval, search, ranking, embeddings, evaluation maturity, product ML experience, availability, and logistics.

Differentiator: it does not rank by raw AI keyword count. Skills are trusted only when backed by career history, duration, endorsements, assessments, and behavioral signals.

Output: a validator-compliant CSV with candidate_id, rank, score, and fact-grounded reasoning for every selected candidate.

Key JD requirements: 5-9 years of hands-on production ML engineering, strong Python, retrieval/ranking ownership, vector or hybrid search, ranking evaluation, and product-shipping mindset.

High-value candidate evidence: shipped search/recommendation/ranking systems, embedding pipelines, FAISS/Milvus/Qdrant/Pinecone/Weaviate/OpenSearch, NDCG/MRR/MAP, A/B tests, scale and latency work.

Behavioral relevance: recent activity, recruiter response rate, response speed, open-to-work status, interview completion, notice period, GitHub signal, recruiter saves/views.

Fit beyond keywords: the ranker reads role history and descriptions, distinguishes product ML builders from keyword stuffers, and down-weights stale or suspicious profiles.

Retrieval/scoring pass: stream candidates.jsonl, extract text and structured features, compute a weighted score, and keep a deterministic top-100 heap.

Algorithms used: interpretable feature engineering, regex-backed career evidence extraction, weighted scoring, behavioral modifier, logistics adjustment, and trap penalties.

Signal fusion: title fit, experience-band fit, retrieval/ranking evidence, evaluation evidence, production evidence, trusted skills, product-company signal, behavior, and location are combined.

Final score: adjusted evidence score is passed through a monotonic 0-1 calibration, then sorted by score descending and candidate_id ascending for deterministic ties.

Reasoning is generated from facts already present in the candidate record: title, years, location, relevant evidence category, response rate, recency, notice period, and concerns.

Hallucination prevention: the generator never invents employers, degrees, skills, or projects. It describes evidence categories only after the scorer found those signals.

Low-quality profile handling: weak expert claims, low skill duration, zero endorsements, stale activity, poor response behavior, and long notice period reduce score.

Suspicious profile handling: severe experience-year mismatches, career duration inconsistencies, non-technical keyword stuffing, off-domain AI, and services-only careers are penalized.

1. Input: load candidates.jsonl or candidates.jsonl.gz and the fixed Senior AI Engineer JD interpretation encoded in the scorer.
2. Feature extraction: parse profile, career history, skills, education, certifications, languages, and Redrob behavioral signals.
3. Score: combine JD evidence, trusted skills, behavioral availability, logistics, and validation penalties.
4. Select: maintain top 100 in memory, sort deterministically, and generate concise recruiter-facing reasoning.
5. Output: write outputs/submission.csv and outputs/audit_top100.json, then validate with validate_submission.py.

CLI entry point: rank.py

Core engine: src/ranker.py

Input layer: streaming JSONL reader with gzip support.

Feature layer: title, text-pattern, skill-trust, product-company, behavior, logistics, and consistency features.

Scoring layer: weighted ranker with engagement gate and trap penalties.

Output layer: top-100 CSV, audit JSON, and fact-grounded reasoning.

Complexity: O(N log 100) over 100,000 candidates, with only the top heap and current record in memory.

Full dataset run completed locally in about 3 minutes 19 seconds on CPU, below the 5-minute challenge limit.

The generated submission passed the official validator: exactly 100 rows, valid candidate IDs, unique ranks, non-increasing scores, and UTF-8 CSV format.

Top-100 quality audit: 95/100 India-based candidates, 0 services-only careers, 0 stale profiles, and no non-technical keyword-stuffer titles.

Top title mix: Recommendation Systems Engineer (18), Applied ML Engineer (13), Machine Learning Engineer (12), Search Engineer (11).

Top examples:

- #1 CAND_0077337: Staff Machine Learning Engineer, 7.0 yrs, Kochi, Kerala, score 0.9995
- #2 CAND_0018499: Senior Machine Learning Engineer, 7.2 yrs, Noida, Uttar Pradesh, score

Python 3.11: selected for portable, reproducible ranking under the CPU/no-network constraint.

Standard library only for ranking: json, csv, gzip, re, heapq, argparse, datetime, math, dataclasses, pathlib.

ReportLab: used only for generating the explanatory PDF deck; it is not required by the ranking pipeline.

No hosted LLM APIs, no GPU, no vector database, no pandas, no scikit-learn, no sentence-transformers, and no network calls during ranking.

This stack was selected because the evaluation rewards reproducibility, latency discipline, explainability, and defensible engineering tradeoffs.

GitHub repo: https://github.com/SavioMohan1/redrob-intelligent-candidate-discovery

Ranked output file: outputs/submission.csv

PDF deck: output/pdf/redrob_intelligent_candidate_discovery_template_filled.pdf

Reproduce command: python rank.py --candidates ./candidates.jsonl --out ./outputs/submission.csv

Validation command: python validate_submission.py ./outputs/submission.csv

Main implementation files: rank.py, src/ranker.py, README.md, docs/approach.md.

Sandbox/demo link: to be added in submission_metadata.yaml after deployment.

Thank You

The solution is designed to rank candidates the way a strong recruiter would: evidence first, behavior-aware, explainable, and reproducible under real-world constraints.

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU